

COMPSCI 589 Homework 4 - Spring 2026

Due **April 30**, 11:55 pm Eastern Time

Instructions

- This homework assignment consists of a programming part.
- While you may discuss problems with your peers (e.g., to discuss high-level approaches), you must answer the questions on your own and implement all solutions independently. In your submission, do explicitly list all students with whom you discussed this assignment.
- Submissions must be typed. You must use L^AT_EX to prepare your submission. Handwritten and scanned submissions will not be accepted.
- The written report, including your answers, plots, and analysis, should be submitted on Gradescope as a PDF via the appropriate submission link. The source code should be submitted as a single .zip file through the separate programming assignment link on Gradescope.
- We strongly encourage using Python 3 for your homework code. You may use other languages. In either case, you *must* provide clear instructions for running your code and reproducing your experiments.
- You *may not* use any machine learning-specific libraries in your code, e.g., TensorFlow, PyTorch, or machine learning algorithms implemented in scikit-learn (though you *may* use other functions provided by this library, such as one that splits a dataset into training and testing sets). You may also use libraries such as numpy and matplotlib. If you are not certain whether a specific library is allowed, do ask us.
- All submissions will be checked for plagiarism using two independent plagiarism-detection tools. Renaming variable or function names, moving code within a file, etc., are all strategies that *do not* fool the plagiarism-detection tools we use. Your code will also be compared with submissions from your classmates and with publicly available implementations online.
- If you get caught, all penalties mentioned in the syllabus *will* be applied—which may include directly failing the course with a letter grade of “F”.

-
- The automated system will not accept assignments after 11:55 pm on April 30.
 - The TeX file for this homework (which you should use to write your solutions in L^AT_EX), as well as the datasets you will investigate and two reference output files you can use to debug your implementation, are available on Canvas under “Homework #4 - Supporting Files”. See below for more details.

Programming Assignment [100 Points, total]

In this homework, you will be implementing the backpropagation algorithm to train a neural network. **Notice that you may not use existing machine learning code for this problem: you must implement the learning algorithm entirely on your own and from scratch.**

In this assignment, you will:

1. Implement the backpropagation algorithm to train a neural network. Your implementation should support networks with an *adjustable* number of layers and neurons. For example, it should support training a neural network with two hidden layers, the first containing 5 neurons and the second containing 3 neurons, or a neural network with three hidden layers, each containing 4 neurons.
2. Implement the regularization mechanism discussed in class, which will be used here to help mitigate overfitting.
3. Evaluate your neural network using the *stratified cross-validation* strategy that you implemented as part of a previous homework. We recommend using $k = 5$ folds. Of these, $k - 1$ folds will be used to train the neural network. The remaining fold (the one not used during training) will be used to evaluate the network's performance. You will then repeat this process, generating and evaluating k neural networks. The final performance is the average of the performances computed across the k folds.
4. Evaluate the performance of your neural network on two datasets as a function of three design decisions: the number of layers in the network, the number of neurons in each layer, and the regularization parameter.

Each student is free to decide whether to implement the “standard” (non-vectorized) version of backpropagation or the vectorized version of backpropagation; that is, the version where all quantities relevant to training, such as predicted outputs, the value of the cost function J , and gradients, are computed using matrix multiplication operations. *We strongly recommend that students complete this assignment using vectorization techniques.* This will significantly reduce the time required to train your neural network and will also lead to a more straightforward implementation of the backpropagation algorithm.

We are providing students with *reference output files* that describe, for two simple neural networks, *all* the relevant quantities computed by a correct implementation of the backpropagation algorithm, step by step. **These are intended to help you debug your implementation of the backpropagation algorithm.** These files describe, for example, the activation of each neuron when the network is presented with a particular training instance, the final output/prediction of the network for that instance, the δ value associated with each neuron, and the corresponding gradients of each weight in the network. Students *should* use the information in these files to ensure that the quantities computed by their implementation of backpropagation match the expected values produced by a correct implementation of this learning algorithm. For example, if the gradients computed by a student's implementation do not match the expected values, the student should be able to quickly identify the first point during the execution of the algorithm where one of the quantities computed by their solution differs from the correct/expected one. This should facilitate debugging. More information about the *reference output files* can be found in Section 1.

Deliverables and Experiments

The main objective of this homework is to study how the performance of a trained neural network is affected by *(i)* the neural network architecture (i.e., the number of layers and neurons) and *(ii)* the regularization parameter.

You should first verify the correctness of your implementation by comparing the outputs it produces with the step-by-step examples we provide (see Section 1). After that, you will analyze two datasets. **Both datasets are included in the homework’s “Supporting Files” zip file on Canvas.**

The two datasets you will investigate are the following:

(1) **The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset.** The goal here is to train a classifier capable of predicting whether a breast mass is malignant or benign, based on features computed from a digitized image of a breast mass. The dataset is composed of 569 instances. Each instance is described by 30 *numerical* attributes, and there are two classes.

(2) **The Loan Eligibility Prediction Dataset.** Here, the goal is to automatically (and accurately) predict whether a given person should qualify for a loan. Each instance is described by 11 attributes, including the gender of the applicant, whether the applicant is married, their income, information about their credit history, etc. The binary target class to be predicted is whether a given applicant’s request for a loan was approved or not. Notice that this dataset contains 7 *categorical* attributes and 4 *numerical* attributes. There is a total of 480 instances in the dataset.

★ Note that some of the datasets above include **both** categorical and numerical attributes. Algorithms such as neural networks (and others) require numerical inputs. The standard way to convert categorical inputs into numerical form is to use *one-hot encoding*. For a quick and high-level introduction to one-hot encoding, as well as examples on how to use Scikit’s libraries to perform this type of conversion, please visit this [website](#).

Important: When implementing your neural network, do not forget to add a *bias* input to each neuron, and make sure that updates to bias weights are handled correctly if regularization is used. For more information, see the slides from the corresponding lecture. Also, make sure to properly normalize the attributes of the training and test instances whenever necessary.

Important: You are free to choose between two simple stopping criteria. You may, for instance, (1) stop if, after presenting all training instances to the network and updating its weights, the cost function J improves by less than some small user-adjusted constant ε ; or (2) stop after a constant, pre-defined number m of iterations, where each iteration consists of presenting all instances in the training set to the network, computing gradients, and updating the network’s weights. You are free to choose which criterion to use.

In all questions below, do not forget to explicitly state which criterion you used and the corresponding hyperparameter (i.e., ε or m). We encourage you to try different settings to identify the one that produces the best possible performance for your algorithm.

1 Correctness Verification

1. You will first need to verify that your implementation of backpropagation is correct; i.e., that the gradients it computes are accurate. To do this, we are providing students with **two reference output files**: *backprop_example1.txt* and *backprop_example2.txt*. Each file describes a specific neural network architecture and a minimal training set. It also shows, in detail, all intermediate quantities computed by the backpropagation algorithm, which you should compare with those produced by your implementation. These quantities include the activation of each neuron when the network is presented with a given input, the final predicted output (and the corresponding expected output), the network's error/cost function J , the δ values for each neuron, and the gradients of all weights.
2. You *must* ensure that your implementation of backpropagation is correct. To do so, you should write a function capable of reproducing the quantities described in each of the two reference output files. This function should produce textual output (e.g., using simple `print` statements) indicating all relevant quantities computed by your implementation, similar to how they are presented in the provided reference output files.
3. As part of your solution to this assignment, you *must* allow us to call this function; that is, the function that demonstrates that your code can reproduce the outputs presented in *backprop_example1.txt* and *backprop_example2.txt*, so that we can verify the correctness of your implementation. The output you generate does not need to be identical to that shown in the reference output files, but it should include at least, for each training instance and for both datasets, the activation of each neuron; the final predicted output of the network; the expected output; the cost J associated with that instance; the δ values of each neuron and the gradients of all weights after the network is presented with that training instance; and the final regularized gradients after the backpropagation algorithm has processed all instances in the training set.

You should then:

- (i) Include in your report clear instructions describing how we (the instructor, staff, and TAs) can run the function you implemented to demonstrate the correctness of your backpropagation algorithm; that is, the function that produces all the results and quantities described above.
- (ii) Present in your report the textual output produced by this function for both provided reference output files. This part of the assignment is extremely important, and a non-trivial portion of your final grade will depend on whether we can verify that you implemented the backpropagation training algorithm correctly.

1. Open the Python file `Homework 4 - Neural Network.ipynb`
2. Ensure all required libraries are installed
3. Ensure you run the code for the following functions - `sigmoid`, `forward_propagation`, `cost_function`, `back_propagation`, `regularized_gradients`
4. For Back Propagation example 1, Theta values for layer 1 and layer 2 are taken from the example file.
The list `J_list` and `all_instance_gradients` are initialized as empty list.
The parameter `lambda` is assigned as 0 as given in the example file.
The input variable `X` and output variable `Y` values are assigned based on the value provided in the example file.
5. Run the script for Backpropagation example 1 using:

```
#### Back propagation example 1 code ####
import numpy as np
Theta_layer1 = np.array([[0.40000,  0.10000],
                          [0.30000,  0.20000]])

Theta_layer2 = np.array([[0.70000,  0.50000, 0.60000]])
```

```

Thetas = [Theta_layer1, Theta_layer2]
J_list = []
lambda_ = 0
all_instance_gradients = []

X = [np.array([0.13]), np.array([0.42])]
Y = [np.array([0.90]), np.array([0.23])]

for i, (x, y) in enumerate(zip(X, Y)):
    result, J = forward_propagation(x, y, Thetas, i+1, verbose = True)
    J_list.append(float(J))
    gradient = back_propagation(result, y, Thetas, i+1, verbose = True)
    all_instance_gradients.append(gradient)

cost_function(J_list, Thetas, lambda_, verbose = True)
regularized_gradients(all_instance_gradients, Thetas, lambda_, verbose = True);

```

6. The Back Propagation example 1 code will produce the following results which match the expected output provided in the example 1 file and verifies the correctness of the implementation:

```

Processing training instance 1
Forward propagating the input [0.13000]
a1: [1.00000 0.13000]
z2: [0.41300 0.32600]
a2: [1.00000 0.60181 0.58079]
z3: [1.34937]
a3: [0.79403]
f(x): [0.79403]
Predicted output for instance 1: [0.79403]
Expected output for instance 1: [0.90000]
Cost, J, associated with instance 1: 0.366

Computing gradients based on training instance 1:
Delta3: [-0.10597]
Delta2: [-0.01270 -0.01548]
Gradients of Theta1 based on training instance 1:
[[-0.01270 -0.00165]
 [-0.01548 -0.00201]]
Gradients of Theta2 based on training instance 1:
[[-0.10597 -0.06378 -0.06155]]
Processing training instance 2
Forward propagating the input [0.42000]
a1: [1.00000 0.42000]
z2: [0.44200 0.38400]
a2: [1.00000 0.60874 0.59484]
z3: [1.36127]
a3: [0.79597]
f(x): [0.79597]
Predicted output for instance 2: [0.79597]
Expected output for instance 2: [0.23000]
Cost, J, associated with instance 2: 1.276

Computing gradients based on training instance 2:
Delta3: [0.56597]

```

```

Delta2: [0.06740 0.08184]
Gradients of Theta1 based on training instance 2:
[[0.06740 0.02831]
 [0.08184 0.03437]]
Gradients of Theta2 based on training instance 2:
[[0.56597 0.34452 0.33666]]
Final (regularized) cost, J, based on the complete training set:0.82098
Final regularized gradients of Theta1:
[[0.02735 0.01333]
 [0.03318 0.01618]]
Final regularized gradients of Theta2:
[[0.23000 0.14037 0.13756]]

```

7. For Back Propagation example 2, Theta values for layer 1, layer 2 and layer 3 are taken from the example file.

The list `J_list` and `all_instance_gradients` are initialized as empty list.

The parameter `lambda` is assigned as `0.250` as given in the example file.

The input variable `X` and output variable `Y` values are assigned based on the value provided in the example file.

8. Run the script for Backpropagation example 2 using:

```

#### Back propagation example 2 code ####
Theta_layer1 = np.array([[0.42000, 0.15000, 0.40000],
                        [0.72000, 0.10000, 0.54000],
                        [0.01000, 0.19000, 0.42000],
                        [0.30000, 0.35000, 0.68000]])

Theta_layer2 = np.array([[0.21000, 0.67000, 0.14000, 0.96000, 0.87000],
                        [0.87000, 0.42000, 0.20000, 0.32000, 0.89000],
                        [0.03000, 0.56000, 0.80000, 0.69000, 0.09000]])

Theta_layer3 = np.array([[0.04000, 0.87000, 0.42000, 0.53000],
                        [0.17000, 0.10000, 0.95000, 0.69000]])

Thetas = [Theta_layer1, Theta_layer2, Theta_layer3]
J_list = []
lambda_ = 0.250
all_instance_gradients = []

X = [np.array([0.32000, 0.68000]), np.array([0.83000, 0.02000])]
Y = [np.array([0.75000, 0.98000]), np.array([0.75000, 0.28000])]

for i, (x, y) in enumerate(zip(X, Y)):
    result, J = forward_propagation(x, y, Thetas, i+1, verbose = True)
    J_list.append(float(J))
    gradient = back_propagation(result, y, Thetas, i+1, verbose = True)
    all_instance_gradients.append(gradient)

cost_function(J_list, Thetas, lambda_, verbose = True)
regularized_gradients(all_instance_gradients, Thetas, lambda_, verbose = True);

```

9. The Back propagation example 2 code will produce the following results which match the expected output provided in the example 2 file and verifies the correctness of the implementation:

```
Processing training instance 1
Forward propagating the input [0.32000 0.68000]
a1: [1.00000 0.32000 0.68000]
z2: [0.74000 1.11920 0.35640 0.87440]
a2: [1.00000 0.67700 0.75384 0.58817 0.70566]
z3: [1.94769 2.12136 1.48154]
a3: [1.00000 0.87519 0.89296 0.81480]
z4: [1.60831 1.66805]
a4: [0.83318 0.84132]
f(x): [0.83318 0.84132]
Predicted output for instance 1: [0.83318 0.84132]
Expected output for instance 1: [0.75000 0.98000]
Cost, J, associated with instance 1: 0.791

Computing gradients based on training instance 1:
Delta4: [ 0.08318 -0.13868]
Delta3: [ 0.00639 -0.00925 -0.00779]
Delta2: [-0.00087 -0.00133 -0.00053 -0.00070]
Gradients of Theta1 based on training instance 1:
[[-0.00087 -0.00028 -0.00059]
 [-0.00133 -0.00043 -0.00091]
 [-0.00053 -0.00017 -0.00036]
 [-0.00070 -0.00022 -0.00048]]
Gradients of Theta2 based on training instance 1:
[[ 0.00639  0.00433  0.00482  0.00376  0.00451]
 [-0.00925 -0.00626 -0.00698 -0.00544 -0.00653]
 [-0.00779 -0.00527 -0.00587 -0.00458 -0.00550]]
Gradients of Theta3 based on training instance 1:
[[ 0.08318  0.07280  0.07427  0.06777]
 [-0.13868 -0.12138 -0.12384 -0.11300]]
Processing training instance 2
Forward propagating the input [0.83000 0.02000]
a1: [1.00000 0.83000 0.02000]
z2: [0.55250 0.81380 0.17610 0.60410]
a2: [1.00000 0.63472 0.69292 0.54391 0.64659]
z3: [1.81696 2.02468 1.37327]
a3: [1.00000 0.86020 0.88336 0.79791]
z4: [1.58228 1.64577]
a4: [0.82953 0.83832]
f(x): [0.82953 0.83832]
Predicted output for instance 2: [0.82953 0.83832]
Expected output for instance 2: [0.75000 0.28000]
Cost, J, associated with instance 2: 1.944

Computing gradients based on training instance 2:
Delta4: [0.07953 0.55832]
Delta3: [0.01503 0.05809 0.06892]
Delta2: [0.01694 0.01465 0.01999 0.01622]
Gradients of Theta1 based on training instance 2:
[[0.01694 0.01406 0.00034]
 [0.01465 0.01216 0.00029]
 [0.01999 0.01659 0.00040]
 [0.01622 0.01346 0.00032]]
```

```

Gradients of Theta2 based on training instance 2:
[[0.01503 0.00954 0.01042 0.00818 0.00972]
 [0.05809 0.03687 0.04025 0.03160 0.03756]
 [0.06892 0.04374 0.04775 0.03748 0.04456]]
Gradients of Theta3 based on training instance 2:
[[0.07953 0.06841 0.07025 0.06346]
 [0.55832 0.48027 0.49320 0.44549]]
Final (regularized) cost, J, based on the complete training set:1.90351
Final regularized gradients of Theta1:
[[0.00804 0.02564 0.04987]
 [0.00666 0.01837 0.06719]
 [0.00973 0.03196 0.05252]
 [0.00776 0.05037 0.08492]]
Final regularized gradients of Theta2:
[[0.01071 0.09068 0.02512 0.12597 0.11586]
 [0.02442 0.06780 0.04164 0.05308 0.12677]
 [0.03056 0.08924 0.12094 0.10270 0.03078]]
Final regularized gradients of Theta3:
[[0.08135 0.17935 0.12476 0.13186]
 [0.20982 0.19195 0.30343 0.25249]]

```

We strongly recommend that you only start working on the next part of this assignment *after* you have verified that your implementation of backpropagation is correct. Otherwise, it may be hard, or even impossible, for your neural network to properly learn solutions to the proposed problems.

The two reference output files we provide (*backprop-example1.txt* and *backprop-example2.txt*) are structured as follows:

- The first line of each file specifies the regularization parameter, λ , that should be used. The next line describes the network structure, that is, the number of neurons in each layer. Recall that the first layer corresponds to the network's input layer and the last layer corresponds to the output layer. In the notation adopted here, the bias neuron is *not* included in the neuron counts shown in this line.
- Consider, for example, a neural network with two inputs, one hidden layer with 3 neurons, and one output. In this case, the second line of the provided files would define the network's structure as [2 3 1]. Notice, however, that your implementation *should* include bias neurons and bias weights. We do not typically count them when specifying the architecture of a network because it is implicit that they are always used.
- After this section of the file, you will encounter a description of the initial/current weights of the network. Each row represents the weights of a specific neuron in a particular layer; the weights of the i -th neuron are stored in the i -th row of the corresponding table/matrix. Recall that the first weight of each neuron corresponds to the bias weight connected to that neuron.

Consider the following simple neural network as an example:

The weights $\theta^{l=1}$ (Theta1) would be shown as follows:

```
0.4 0.1  
0.3 0.2
```

The weights $\theta^{l=2}$ (Theta2) would be shown as follows:

```
0.7 0.5 0.6
```

After this section of the file, you will find a description of the inputs (x) and expected outputs (y) for each instance in the training set. The subsequent parts of the file show the intermediate quantities computed during forward propagation for each training instance. In particular, it reports the z and a values of each neuron (i.e., their activations), as well as the network's final prediction, $f(x)$, and the cost function J associated with that instance.

The final section of each file presents all intermediate quantities computed during the backpropagation phase: the δ values of each neuron after processing a given training instance, and the gradients of all weights after that instance is processed. It then reports the final regularized gradients of all weights, computed using all training instances. As discussed in class, these gradients are used to update the network's weights so that it, on average, makes better predictions across all training examples in the dataset.

2 Experiments and Analyses

For each dataset, you should:

1. Train a neural network and evaluate it using the stratified cross-validation technique discussed in class. You should train neural networks using different values of the regularization parameter, λ , and different network architectures. You may decide which architectures to evaluate. For example, you might test networks with 1, 2, 3, and 4 hidden layers, and vary the number of neurons per layer, such as 2, 4, 8, and 16.

WDBC Dataset

- (a) For the WDBC dataset, I evaluated models with multiple hidden layers and varying the number of neurons per layer:
 - 1 hidden layer : [30, 2, 1], [30, 4, 1], [30, 6, 1], [30, 8, 1], [30, 12, 1], [30, 16, 1], [30, 24, 1], [30, 32, 1]
 - 2 hidden layers : [30, 2, 4, 1], [30, 2, 16, 1], [30, 4, 8, 1], [30, 4, 16, 1], [30, 8, 16, 1]
 - 3 hidden layers : [30, 4, 8, 16, 1]
 - 4 hidden layers : [30, 2, 2, 2, 2, 1], [30, 2, 4, 8, 16, 1]
- (b) Regularization parameter λ used for the neural network takes the value : [0, 0.1, 0.01]
- (c) The Stratified Cross Validation technique used for evaluation is $K = 5$ (stratified 5-fold cross-validation)
- (d) The stopping criterion used is a constant, pre-defined number of iterations ($m = 1000$)
- (e) The step size used is $\alpha = 0.1$

Loan Dataset

- (a) For the Loan dataset, I evaluated models with multiple hidden layers and varying the number of neurons per layer:
 - 1 hidden layer : [21, 2, 1], [21, 8, 1], [21, 12, 1], [21, 16, 1], [21, 24, 1], [21, 32, 1],
 - 2 hidden layers : [21, 4, 8, 1], [21, 16, 4, 1], [21, 4, 16, 1], [21, 8, 16, 1], [21, 16, 8, 1], [21, 16, 16, 1], [21, 16, 32, 1]
 - 3 hidden layers : [21, 4, 8, 16, 1]
 - 4 hidden layers : [21, 2, 2, 2, 2, 1], [21, 2, 4, 8, 16, 1]
- (b) Regularization parameter λ used for the neural network takes the value : [0, 0.1, 0.01]
- (c) The Stratified Cross Validation technique used for evaluation is $K = 5$ (stratified 5-fold cross-validation)
- (d) The stopping criterion used is a constant, pre-defined number of iterations ($m = 1000$)
- (e) The step size used is $\alpha = 0.1$

For each trained neural network, that is, for each combination of architecture and regularization that you tested, you should measure the resulting model's accuracy and F1-score.

- (a) The Accuracy and F1 score for the WDBC and Loan dataset were measured for different neural network architectures, varying the number of hidden layers, and neurons per layer
- (b) The results for the WDBC dataset is presented in Table 1 and the results for the Loan dataset is presented in Table 2.

Based on these experiments, you should create, for each dataset and for each of the metrics described above, a table summarizing the results. In particular, report the value of each performance metric for every neural network configuration you tested, on both datasets. **You should test at least 6 neural network architectures on each dataset.**

WDBC Dataset

Table 1: WDBC dataset architectures with performance metrics : Accuracy and F1 Score for every neural network and various values of λ .

Architecture	λ	Accuracy	F1 Score
[30, 2, 1]	0	0.938515	0.953137
[30, 2, 1]	0.1	0.885479	0.917784
[30, 2, 1]	0.01	0.913859	0.935032
[30, 4, 1]	0	0.942085	0.955817
[30, 4, 1]	0.1	0.941993	0.955645
[30, 4, 1]	0.01	0.947303	0.959429
[30, 6, 1]	0	0.952489	0.963162
[30, 6, 1]	0.1	0.943917	0.957138
[30, 6, 1]	0.01	0.94721	0.958767
[30, 8, 1]	0	0.950812	0.961635
[30, 8, 1]	0.1	0.950781	0.961181
[30, 8, 1]	0.01	0.947364	0.958941
[30, 12, 1]	0	0.942024	0.954846
[30, 12, 1]	0.1	0.945594	0.95712
[30, 12, 1]	0.01	0.959631	0.968488
[30, 16, 1]	0	0.96137	0.969918
[30, 16, 1]	0.1	0.945564	0.957436
[30, 16, 1]	0.01	0.94718	0.959027
[30, 24, 1]	0	0.95783	0.966952
[30, 24, 1]	0.1	0.95429	0.964264
[30, 24, 1]	0.01	0.9596	0.968555
[30, 32, 1]	0	0.955998	0.965377
[30, 32, 1]	0.1	0.961308	0.969512
[30, 32, 1]	0.01	0.957768	0.966983
[30, 2, 4, 1]	0	0.713721	0.819784
[30, 2, 4, 1]	0.1	0.748503	0.841546
[30, 2, 4, 1]	0.01	0.684817	0.804067
[30, 2, 16, 1]	0	0.889788	0.922413
[30, 2, 16, 1]	0.1	0.883555	0.921948
[30, 2, 16, 1]	0.01	0.887926	0.920404
[30, 4, 8, 1]	0	0.885848	0.917935
[30, 4, 8, 1]	0.1	0.874044	0.915025
[30, 4, 8, 1]	0.01	0.854636	0.902073

Architecture	λ	Accuracy	F1 Score
[30, 4, 16, 1]	0	0.926033	0.944292
[30, 4, 16, 1]	0.1	0.917491	0.935998
[30, 4, 16, 1]	0.01	0.929788	0.945704
[30, 8, 16, 1]	0	0.95075	0.96131
[30, 8, 16, 1]	0.1	0.949042	0.960751
[30, 8, 16, 1]	0.01	0.938761	0.953148
[30, 4, 8, 16, 1]	0	0.675214	0.797609
[30, 4, 8, 16, 1]	0.1	0.866195	0.909139
[30, 4, 8, 16, 1]	0.01	0.669904	0.794217
[30, 2, 2, 2, 2, 1]	0	0.627426	0.771065
[30, 2, 2, 2, 2, 1]	0.1	0.627426	0.771065
[30, 2, 2, 2, 2, 1]	0.01	0.627426	0.771065
[30, 2, 4, 8, 16, 1]	0	0.627426	0.771065
[30, 2, 4, 8, 16, 1]	0.1	0.627426	0.771065
[30, 2, 4, 8, 16, 1]	0.01	0.627426	0.771065

Loan Dataset

Table 2: Loan dataset architectures with performance metrics : Accuracy and F1 Score for every neural network and various values of λ .

Architecture	λ	Accuracy	F1 Score
[21, 2, 1]	0	0.787656	0.863729
[21, 2, 1]	0.1	0.746223	0.844743
[21, 2, 1]	0.01	0.74603	0.843972
[21, 8, 1]	0	0.797985	0.869739
[21, 8, 1]	0.1	0.806341	0.87499
[21, 8, 1]	0.01	0.799766	0.8704
[21, 12, 1]	0	0.804127	0.872946
[21, 12, 1]	0.1	0.80402	0.87286
[21, 12, 1]	0.01	0.806298	0.874708
[21, 16, 1]	0	0.808339	0.876208
[21, 16, 1]	0.1	0.808121	0.876467
[21, 16, 1]	0.01	0.806168	0.874182
[21, 24, 1]	0	0.810553	0.877318
[21, 24, 1]	0.1	0.806385	0.874275

Architecture	λ	Accuracy	F1 Score
[21, 24, 1]	0.01	0.799896	0.869823
[21, 32, 1]	0	0.805951	0.874708
[21, 32, 1]	0.1	0.806407	0.874889
[21, 32, 1]	0.01	0.804151	0.872309
[21, 4, 8, 1]	0	0.760593	0.852234
[21, 4, 8, 1]	0.1	0.737868	0.839638
[21, 4, 8, 1]	0.01	0.714451	0.82894
[21, 16, 4, 1]	0	0.756382	0.850508
[21, 16, 4, 1]	0.1	0.775132	0.858915
[21, 16, 4, 1]	0.01	0.752259	0.847174
[21, 4, 16, 1]	0	0.754385	0.847584
[21, 4, 16, 1]	0.1	0.742034	0.841462
[21, 4, 16, 1]	0.01	0.799982	0.870921
[21, 8, 16, 1]	0	0.806038	0.874281
[21, 8, 16, 1]	0.1	0.812592	0.878991
[21, 8, 16, 1]	0.01	0.80415	0.873114
[21, 16, 8, 1]	0	0.795752	0.869572
[21, 16, 8, 1]	0.1	0.81038	0.877477
[21, 16, 8, 1]	0.01	0.8081	0.876508
[21, 16, 16, 1]	0	0.806233	0.874661
[21, 16, 16, 1]	0.1	0.806298	0.874655
[21, 16, 16, 1]	0.01	0.80211	0.872316
[21, 16, 32, 1]	0	0.804324	0.873406
[21, 16, 32, 1]	0.1	0.806364	0.874743
[21, 16, 32, 1]	0.01	0.806472	0.874005
[21, 4, 8, 16, 1]	0	0.691683	0.817743
[21, 4, 8, 16, 1]	0.1	0.691683	0.817743
[21, 4, 8, 16, 1]	0.01	0.691683	0.817743
[21, 2, 2, 2, 2, 1]	0	0.691683	0.817743
[21, 2, 2, 2, 2, 1]	0.1	0.691683	0.817743
[21, 2, 2, 2, 2, 1]	0.01	0.691683	0.817743
[21, 2, 4, 8, 16, 1]	0	0.691683	0.817743
[21, 2, 4, 8, 16, 1]	0.1	0.691683	0.817743
[21, 2, 4, 8, 16, 1]	0.01	0.691683	0.817743

Discussion: Discuss (at a high level) what contributed most to improving performance: changing the regularization parameter; adding more layers; using deeper networks with many layers but few neurons per layer; or designing networks with few layers but many neurons per layer. Describe any patterns you may have observed. Also discuss whether there is a point at which constructing and training more “sophisticated”/complex networks, that is, larger networks, no longer improves performance (or even worsens it).

WDBC Dataset:

- (a) Adding 3 hidden layers causes the performance metrics to degrade significantly compared to simpler architecture
- (b) Adding 4 hidden layers, regardless of the number of neurons in each layer causes the performance metrics to drop to the lowest level for all values of λ .
- (c) The performance metrics are highest for architectures with only a single hidden layer, suggesting that simpler models generalize well for the WDBC dataset
- (d) For one hidden layer architecture, the performance is higher for a higher number of neurons (e.g. 16, 24, 32 neurons) in the hidden layer. The model generalizes well for higher number of neurons in single hidden layer architecture
- (e) Changing the regularization parameter λ has a moderate impact on performance metrics
- (f) Adding more hidden layers (e.g., 4 hidden layers) leads to overfitting in the neural networks, as the performance metrics decreases with increasing hidden layers or increasing the model complexity
- (g) Architecture with single hidden layer and a small number of neurons in the hidden layer produce high performance metrics. This suggest that the model does not underfit and is able to generalizes well with simple architecture
- (h) The WDBC dataset is simple and its peak performance is captured by single hidden layer with moderate number of neurons
- (i) Architecture with two hidden layers with few neurons in each layer (e.g., 2 or 4 neurons) in each layer produces below-average performance metrics which may indicate underfitting.
- (j) Architecture with two hidden layers and a larger number of neurons in each layer (e.g., 8 or 16 neurons) generalizes well.
- (k) Training more complex (larger) neural networks, no longer improves performance

Loan Dataset:

- (a) Adding 3 or 4 hidden layers, regardless of the number of neurons in each layer, causes the performance metrics to drop to the lowest levels for all values of λ . The performance stagnate across these architectures irrespective of the number of neurons in the hidden layer in these architectures.
- (b) Adding more hidden layers (e.g., 3 or 4 hidden layers) leads to poor generalization in the neural networks, as the performance metrics stagnate with increasing hidden layers or increasing the model complexity. This indicates diminishing returns with more complex models.
- (c) Changing the regularization parameter λ has a moderate impact on performance metrics
- (d) Architecture with two hidden layers and higher number of neurons in each layer improve the performance metrics of the neural network and generalize well.

- (e) Training more complex (larger) neural networks does not lead to further improvements in performance
- (f) Architecture with single hidden layer and a moderate number of neurons in the hidden layer (e.g., 8, 12, 16, 24 or 32 neurons) produce high performance metrics. This suggests that the model does not underfit and is able to generalize well with simple architecture
- (g) Architecture with two hidden layers and a small number of neurons in each layer exhibit poorer performance compared to those architecture with two hidden layers and higher number of neurons
- (h) Architecture with one hidden layer and a small number of neurons (e.g., 2 neurons) exhibit poorer performance compared to architecture with one hidden layer and higher number of neurons (e.g., 8 or 12 or 16 or 32 neurons). This indicates that higher capacity neural network model improves performance

Based on the analyses above, discuss which neural network architecture you would select if you had to deploy such a classifier in practice. Explain your reasoning.

WDBC Dataset

- (a) I would select the architecture [30, 16, 1] with $\lambda = 0$ if I had to deploy such a classifier in practice for WDBC dataset. This architecture achieves the highest accuracy (0.96137) and F1 score (0.969918) for the WDBC dataset.
- (b) The neural network model generalizes well for a higher number of neurons (e.g., 16 or 32 neurons) in a single hidden layer architecture.
- (c) Architectures with multiple hidden layers and many neurons per hidden layer does not generalize well and do not provide performance improvements

Loan Dataset

- (a) I would select the architecture [21, 8, 16, 1] with $\lambda = 0.1$ if I had to deploy such a classifier in practice for the Loan dataset. This architecture achieves the highest accuracy (0.812592) and F1 score (0.878991) for the Loan dataset.
- (b) The neural network model generalizes well with a higher number of neurons (e.g., 8 and 16 neurons) in a two-hidden-layer architecture.
- (c) Architectures with three or four hidden layers do not generalize well and do not provide additional performance improvements

After identifying the best neural network architecture for each dataset, train it again on the corresponding dataset and create a learning curve in which the y -axis shows the network's performance (J) on a test set and the x -axis shows the number of training samples provided to the network. This graph should illustrate the following idea: after showing 5 training examples to the network, what is its performance J on the test set? After showing 10 training examples, what is its performance J on the test set? If the network's parameters and architecture are well adjusted, the graph should show that performance improves as the number of training examples increases; in particular, that J *decreases* as a function of the number of training instances presented to the network. Please also report the step size value, α , that you used when training this network.

WDBC Dataset

- (a) The step size α used for training this network is 0.1

- (b) The Cost Function J decreases as the number of training instances increases. This indicates that the neural network model performance improves as the number of training data provided increases.
- (c) The Cost Function J decreases rapidly as the training size increases from 5 to 100. For larger training sizes, the cost functions stabilize, indicating the model generalizes well

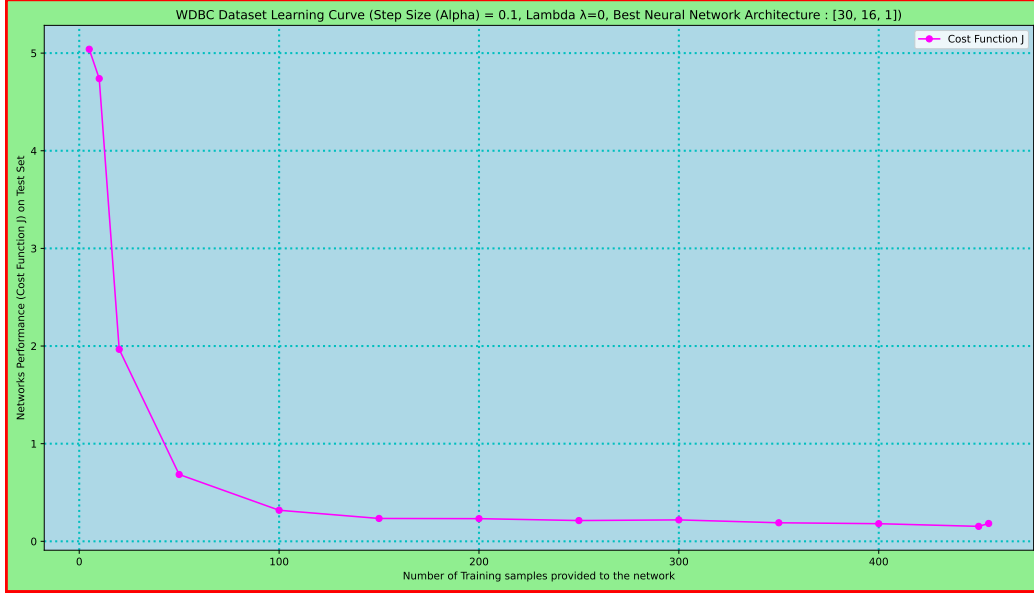


Figure 1: Learning Curve for Neural Network on WDBC Dataset

Loan Dataset

- (a) The step size α used for training this network is 0.1
- (b) The cost function J initially increases as the training size grows from 5 to training size 20, then the cost function decreases for training size 20 to training size 100. Beyond this point, the cost function stabilizes with minor fluctuations for higher training instances
- (c) The initial fluctuation in the cost function may be due to the neural network adapting to diverse training data
- (d) The stabilization of the cost function for larger training sizes indicates that the model generalizes well with more training instances.

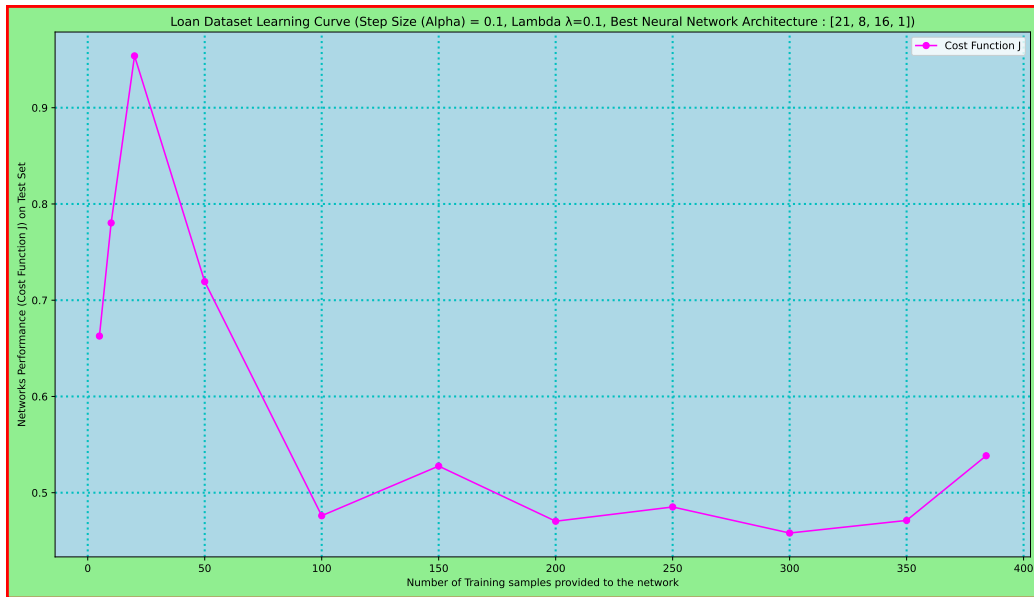


Figure 2: Learning Curve for Neural Network on Loan Dataset

2. We recommend, though this is *not* required, that you train the network using the mini-batch gradient descent approach. You may manually select and adjust the mini-batch size you would like to use. Training a neural network in this way can significantly speed up the training process.

Some Hints (Important!)

1. Initialize the network's weights with small random numbers from -1 to +1, or with random numbers sampled from a Gaussian distribution with zero mean and variance equal to 1.
2. When trying to identify effective network architectures, start by testing networks with few layers and few neurons. For example, you could begin with networks that have just one hidden layer. Increase the number of layers and/or the number of neurons per layer only when necessary.
3. When training a given neural network architecture, try using larger step sizes, α , at first. If you set α too small, the training process may become very slow or may prematurely converge to a bad local minimum, depending on the stopping criterion you use. By contrast, if the network's performance oscillates significantly during training when using large values of α , you may be overshooting. In that case, decrease the value of α . Repeat this process until you find the largest step size that can be used effectively; that is, the largest value that allows the weights to be updated fairly quickly but does not cause instability or divergence.
4. After you identify a value of α that causes the network's performance to improve in a reasonably consistent way, check the performance (value of J) to which the network converges. If it is not satisfactory, your network architecture might be too simple, that is, the network may be underfitting. In that case, try increasing the number of layers and/or the number of neurons per layer. Then repeat the analysis above to evaluate different step sizes, α .

Extra Credit Questions

There are three ways in which you may receive extra credit on this homework.

[QE.1] [Extra Credit Question #1] (13 Points)

Analyze a third dataset: the **Raisins Dataset**. The goal here is to use morphological features extracted from raisin images to classify each instance as either a Kecimen or a Besni raisin variety. The dataset contains 900 instances. Each instance is described by 7 *numerical* features, and the dataset contains two classes. You should present the same analyses and graphs discussed above.

Raisin Dataset

1. For the Raisin dataset, I evaluated models with multiple hidden layers and varying the number of neurons per layer:
 - 1 hidden layer : [7, 2, 1], [7, 16, 1], [7, 4, 1], [7, 8, 1], [7, 12, 1], [7, 24, 1], [7, 32, 1]
 - 2 hidden layers : [7, 8, 16, 1], [7, 4, 8, 1]
 - 3 hidden layers : [7, 4, 8, 16, 1]
 - 4 hidden layers : [7, 2, 2, 2, 2, 1], [7, 2, 4, 8, 16, 1]
2. Regularization parameter λ used for the neural network takes the value : [0, 0.1, 0.01]
3. The Stratified Cross Validation technique used for evaluation is $K = 5$ (stratified 5-fold cross-validation)
4. The stopping criterion used is a constant, pre-defined number of iterations ($m = 1000$)
5. The step size used is $\alpha = 0.1$
6. The Accuracy and F1 score for the Raisin dataset were measured for different neural network architectures, varying the number of hidden layers, and neurons per layer. The results for the Raisin dataset is presented in Table 3
7. I would select the architecture [7, 32, 1] with $\lambda = 0.1$ if I had to deploy such a classifier in practice for the Raisin dataset. This architecture achieves the highest accuracy (0.865556) and F1 score (0.862422) for the Raisin dataset.

Table 3: Raisin dataset architectures with performance metrics : Accuracy and F1 Score for every neural network and various values of λ .

Architecture	λ	Accuracy	F1 Score
[7, 2, 1]	0	0.782222	0.792023
[7, 2, 1]	0.1	0.847778	0.839348
[7, 2, 1]	0.01	0.824444	0.812648
[7, 4, 1]	0	0.835556	0.834478
[7, 4, 1]	0.1	0.862222	0.855128
[7, 4, 1]	0.01	0.845556	0.837461
[7, 8, 1]	0	0.845556	0.83923
[7, 8, 1]	0.1	0.86	0.855549

Architecture	λ	Accuracy	F1 Score
[7, 8, 1]	0.01	0.865556	0.860794
[7, 12, 1]	0	0.852222	0.848055
[7, 12, 1]	0.1	0.863333	0.85927
[7, 12, 1]	0.01	0.863333	0.85967
[7, 16, 1]	0	0.857778	0.854892
[7, 16, 1]	0.1	0.856667	0.852329
[7, 16, 1]	0.01	0.856667	0.852901
[7, 24, 1]	0	0.865556	0.861652
[7, 24, 1]	0.1	0.86	0.856385
[7, 24, 1]	0.01	0.861111	0.858638
[7, 32, 1]	0	0.865556	0.862114
[7, 32, 1]	0.1	0.865556	0.862422
[7, 32, 1]	0.01	0.863333	0.859152
[7, 4, 8, 1]	0	0.845556	0.833902
[7, 4, 8, 1]	0.1	0.84	0.831015
[7, 4, 8, 1]	0.01	0.857778	0.850731
[7, 8, 16, 1]	0	0.851111	0.845597
[7, 8, 16, 1]	0.1	0.854444	0.848284
[7, 8, 16, 1]	0.01	0.841111	0.834372
[7, 4, 8, 16, 1]	0	0.792222	0.78042
[7, 4, 8, 16, 1]	0.1	0.768889	0.749597
[7, 4, 8, 16, 1]	0.01	0.794444	0.786402
[7, 2, 2, 2, 2, 1]	0	0.507778	0.430625
[7, 2, 2, 2, 2, 1]	0.1	0.565556	0.430829
[7, 2, 2, 2, 2, 1]	0.01	0.561111	0.476898
[7, 2, 4, 8, 16, 1]	0	0.693333	0.695573
[7, 2, 4, 8, 16, 1]	0.1	0.431111	0.489923
[7, 2, 4, 8, 16, 1]	0.01	0.486667	0.562976

1. The step size α used for training the dataset and constructing the learning curve is 0.1.
2. The cost function J shows a noticeable decrease between training size 450 to training size 500, and then stabilizes for larger training instances. This indicates the neural network model performance improves with more training instances (500 training instances) and becomes consistent beyond this range.
3. Initially, the model performance deteriorates for training size 5 to training size 450. The cost function remains high from training size 5 to training size 450. The model performance is poor for limited training instances and requires more sample data for effective learning.

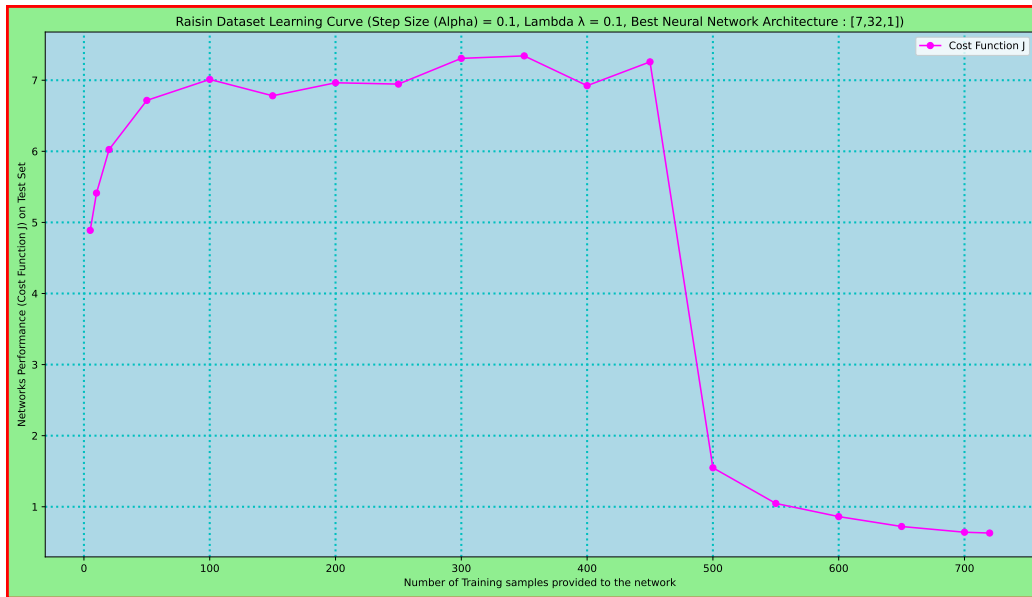


Figure 3: Learning Curve for Neural Network on Raisin Dataset

This dataset can be found in the “*Supporting Files*” zip file for this homework on Canvas.

[QE.2] [Extra Credit Question #2] (13 Points)

Analyze a fourth dataset: the **Titanic Dataset**. On April 15, 1912, the largest passenger liner ever made collided with an iceberg during her maiden voyage. When the Titanic sank, it killed 1502 out of 2224 passengers and crew. One of the reasons the shipwreck resulted in such loss of life was that there were not enough lifeboats for the passengers and crew. Although luck was involved in surviving the sinking, some groups of people were more likely to survive than others. The goal here is to predict whether a given person was likely to survive this tragedy. The dataset contains data for 887 real passengers of the Titanic. Each row represents one person. The columns describe different attributes of the person, including whether they survived (the class/label), their age, their passenger class, sex, and the fare they paid. Notice that this dataset combines *both numerical and categorical attributes*. You should present the same analyses and graphs discussed above.

Titanic Dataset

- For the Titanic dataset, I evaluated models with multiple hidden layers and varying the number of neurons per layer:
 - 1 hidden layer : [7, 2, 1], [7, 16, 1], [7, 4, 1], [7, 8, 1], [7, 12, 1], [7, 24, 1], [7, 32, 1]
 - 2 hidden layers : [7, 8, 16, 1], [7, 16, 8, 1], [7, 4, 8, 1], [7, 4, 16, 1], [7, 8, 4, 1], [7, 16, 4, 1]
 - 3 hidden layers : [7, 4, 8, 16, 1]
 - 4 hidden layers : [7, 2, 2, 2, 2, 1], [7, 2, 4, 8, 16, 1]
- Regularization parameter λ used for the neural network takes the value : [0, 0.1, 0.01]

3. The Stratified Cross Validation technique used for evaluation is $K = 5$ (stratified 5-fold cross-validation)
4. The stopping criterion used is a constant, pre-defined number of iterations ($m = 1000$)
5. The step size used is $\alpha = 0.1$
6. The Accuracy and F1 score for the Titanic dataset were measured for different neural network architectures, varying the number of hidden layers, and neurons per layer. The results for the Titanic dataset is presented in Table 4
7. I would select the architecture $[7, 32, 1]$ with $\lambda = 0.01$ if I had to deploy such a classifier in practice for the Titanic dataset. This architecture achieves the highest accuracy (0.7903) and F1 score (0.719042) for the Titanic dataset.

Table 4: Titanic dataset architectures with performance metrics : Accuracy and F1 Score for every neural network and various values of λ .

Architecture	λ	Accuracy	F1 Score
[7, 2, 1]	0	0.788053	0.712015
[7, 2, 1]	0.1	0.789183	0.712955
[7, 2, 1]	0.01	0.7858	0.709055
[7, 4, 1]	0	0.785806	0.710478
[7, 4, 1]	0.1	0.789208	0.711146
[7, 4, 1]	0.01	0.786949	0.71243
[7, 8, 1]	0	0.785812	0.710441
[7, 8, 1]	0.1	0.785704	0.710051
[7, 8, 1]	0.01	0.786942	0.711845
[7, 12, 1]	0	0.785774	0.708944
[7, 12, 1]	0.1	0.786936	0.711089
[7, 12, 1]	0.01	0.785831	0.711124
[7, 16, 1]	0	0.788047	0.713334
[7, 16, 1]	0.1	0.786904	0.712256
[7, 16, 1]	0.01	0.786929	0.709393
[7, 24, 1]	0	0.789221	0.713017
[7, 24, 1]	0.1	0.785819	0.710885
[7, 24, 1]	0.01	0.790288	0.716307
[7, 32, 1]	0	0.788078	0.712466
[7, 32, 1]	0.1	0.792586	0.716288
[7, 32, 1]	0.01	0.7903	0.719042
[7, 4, 8, 1]	0	0.759798	0.568135
[7, 4, 8, 1]	0.1	0.784657	0.707853

Architecture	λ	Accuracy	F1 Score
[7, 4, 8, 1]	0.01	0.785825	0.71021
[7, 4, 16, 1]	0	0.785755	0.708023
[7, 4, 16, 1]	0.1	0.785787	0.709805
[7, 4, 16, 1]	0.01	0.789196	0.713325
[7, 8, 4, 1]	0	0.758694	0.594994
[7, 8, 4, 1]	0.1	0.763296	0.638591
[7, 8, 4, 1]	0.01	0.788028	0.693808
[7, 16, 4, 1]	0	0.78799	0.708575
[7, 16, 4, 1]	0.1	0.786904	0.71154
[7, 16, 4, 1]	0.01	0.786987	0.711528
[7, 16, 8, 1]	0	0.78698	0.710603
[7, 16, 8, 1]	0.1	0.785787	0.710207
[7, 16, 8, 1]	0.01	0.788059	0.713407
[7, 8, 16, 1]	0	0.788098	0.713383
[7, 8, 16, 1]	0.1	0.785774	0.709872
[7, 8, 16, 1]	0.01	0.788015	0.712434
[7, 4, 8, 16, 1]	0	0.712588	0.418585
[7, 4, 8, 16, 1]	0.1	0.680886	0.278215
[7, 4, 8, 16, 1]	0.01	0.789221	0.713244
[7, 2, 2, 2, 2, 1]	0	0.614435	0.0
[7, 2, 2, 2, 2, 1]	0.1	0.614435	0.0
[7, 2, 2, 2, 2, 1]	0.01	0.614435	0.0
[7, 2, 4, 8, 16, 1]	0	0.614435	0.0
[7, 2, 4, 8, 16, 1]	0.1	0.614435	0.0
[7, 2, 4, 8, 16, 1]	0.01	0.614435	0.0

1. The step size α used for training the dataset and constructing the learning curve is 0.1.
2. The cost function J decreases between training size 10 to training size 20 and then stabilizes for larger training sizes
3. The neural network model is able to learn effectively from a small number of training instances and generalizes well as more training instances are provided.

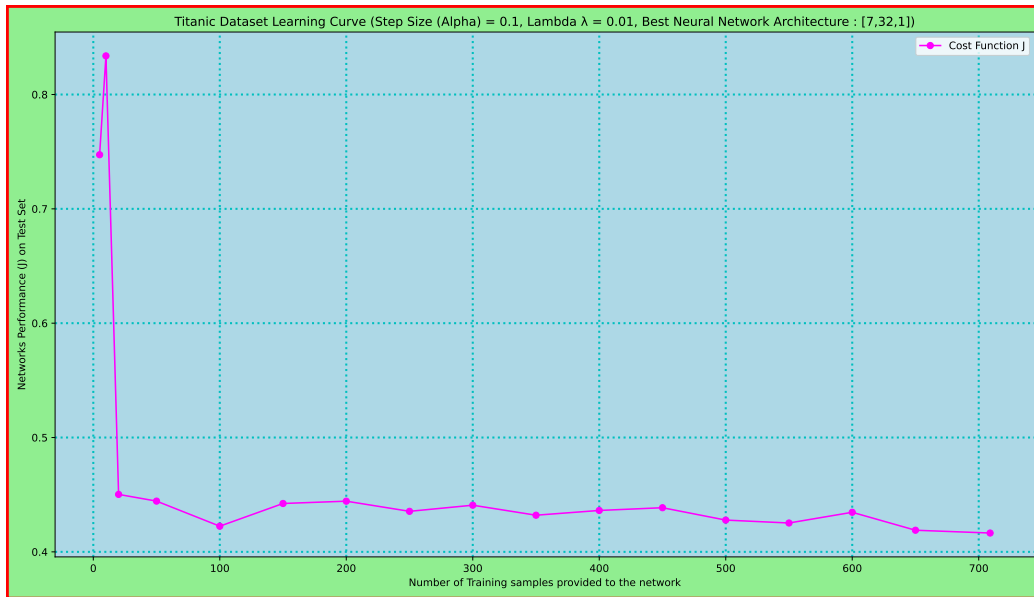


Figure 4: Learning Curve for Neural Network on Titanic Dataset

This dataset can be found in the “*Supporting Files*” zip file for this homework on Canvas.

[QE.3] [Extra Credit Question #3] (13 Points)

Implement the method discussed in class that allows you to *numerically* verify the gradients computed by your implementation of backpropagation. This will help you further ensure that all gradients computed by your implementation are correct, so that you can confidently trust your neural network training procedure. In your report, present the estimated gradients for the two neural networks described in the provided reference output files. First, estimate and report the gradients using $\epsilon = 0.1$, and then using $\epsilon = 0.000001$. If you choose to complete this extra-credit task, please include the corresponding source code with your Gradescope submission.